Indian Institute of Science, Bengaluru

Robert Bosch Centre for Cyber-Physical Systems

Stabilizing Constrained Diffusion Execution Pipelines for Continuous Robotic Navigation via Inverse Dynamics and Pure-Pursuit Control

Internship Report

Author: Maharnab Das

Programme: Research Internship

Supervisor: Dr. Pradipta Biswas, Associate Professor

Duration: May 2026 – June 2026

Submitted in partial fulfilment of the requirements of the internship programme at the Indian Institute of Science, Bengaluru.

July 1, 2026

Declaration

I hereby declare that this report, entitled *"Stabilizing Constrained Diffusion Execution Pipelines for Continuous Robotic Navigation via Inverse Dynamics and Pure-Pursuit Control,"* constitutes an authentic record of original work undertaken during my internship at the Robert Bosch Centre for Cyber-Physical Systems, Indian Institute of Science, Bengaluru, under the supervision of **Dr. Pradipta Biswas, Associate Professor**.

No part of this report has previously been submitted for the award of any degree or diploma at this or any other institution. All sources of information consulted in its preparation have been duly cited and acknowledged. Ideas, derivations, and code not originating with the author are explicitly identified as such within the text.

_____ _____

IISc Bengaluru

Acknowledgements

I record my sincere gratitude to **Dr. Pradipta Biswas, Associate Professor**, Indian Institute of Science, Bengaluru, for the expert guidance, critical feedback, and sustained encouragement extended over the course of this internship.

I thank the members of the Multipurpose Lab, CPS Department for several substantive discussions on the control-theoretic aspects of the Pure-Pursuit pipeline and on the practical questions raised by deploying the diffusion model in closed loop.

The computational resources made available through the facility at IISc are gratefully acknowledged; the extended trajectory-rollout experiments reported here would not have been possible without them.

I am also grateful to colleagues in the laboratory for their time and useful suggestions throughout the course of this work.

Abstract

Diffusion-based trajectory planners have demonstrated a strong capacity to capture multimodal trajectory distributions from offline data, which has motivated their increasing adoption in offline reinforcement learning (RL). Considerably less attention has been paid to the gap between *planning quality*, as assessed by standard offline metrics, and *execution stability*, which becomes apparent only once a generated plan is tracked by a controller operating under continuous, closed-loop conditions.

This report documents three such failures, encountered during the deployment of a **Constrained Diffuser** model on the `maze2d-large-v1` benchmark, together with the remediations developed for each. A two-component execution layer—an **Inverse Dynamics Model (IDM)** coupled with an **Adaptive Pure-Pursuit controller**—was constructed to convert the planner's waypoint sequences into continuous robotic actions. Three architectural variants were evaluated under this unified pipeline: *Unconstrained Diffusion* (UCD), *Primal-Dual Constrained Diffusion* (PD-CD), and *Projected Gradient Constrained Diffusion* (PG-CD).

The three failure modes identified, and the corresponding remediations, are as follows.

1. **Geometric corner-cutting and pursuit-vector collapse (FM-1).** A lookahead of d_{la}=0.20 produced repeated obstacle-boundary violations, whereas d_{la}=0.10 induced 86 or more trajectory stalls through collapse of the pursuit vector. Analysis of the feasibility interval $(\epsilon_{\text{track}}, r_{\text{obs}}) = (0.10, 0.50)$ identifies $d_{\text{la}} = 0.15$ as a suitable operating point.

2. **Limit-cycle oscillation from EMA phase lag (FM-2).** An Exponential Moving Average (EMA) filter situated inside the closed-loop feedback path introduced sufficient phase lag to destabilise tracking, manifesting as sustained underdamped lateral weaving. Removing the EMA from the feedback path and introducing a curvature-aware adaptive lookahead schedule (d_{la}=0.30 on straight segments, 0.15 through curves) reduced the number of execution steps required by 15.5% (490 $\rightarrow$ 414).

3. **Fixed-horizon distance burning and homotopy collapse (FM-3).** The fixed temporal budget of T=384 steps obliges the score function to fabricate path length, typically through looping and retracing, whenever the physical geodesic

is comparatively short. For PD-CD, the dynamic Lagrangian multipliers were further observed to reverse the homotopy class of the planned solution, routing the trajectory through the topologically incorrect corridor. A single post-hoc application of the Douglas–Peucker simplification algorithm, performed prior to execution, removes the spurious loops and restores the correct corridor, enabling the trajectories to reach the goal successfully.

Once the stabilisation pipeline is applied, the empirical results challenge the theoretical premise of the Constrained Diffuser framework. While explicit constraint architectures (PG-CD and PD-CD) successfully avoid the specific unmodeled obstacles they are analytically constrained against, they fail to generalize safety to the implicitly learned maze boundaries. By rigidly projecting trajectories to avoid the analytical obstacle, PG-CD inadvertently forces the robot closer to the implicit maze walls, leading to higher average boundary violations than the Unconstrained Diffusion (UCD) baseline. We conclude that while explicit score-space projections are mathematically sound, deploying them safely in continuous environments requires differentiating between analytical collision constraints and implicitly learned geometric boundaries. None of the three fixes—optimal lookahead scaling, the EMA-bypass protocol, and Douglas–Peucker path shortening—depends on the particular diffusion architecture in use, and each should transfer directly to other fixed-horizon planners.

Keywords: Constrained Diffusion, Inverse Dynamics Model, Pure-Pursuit Control, Cyber-Physical Systems, Limit Cycle Oscillation, Homotopy Collapse, Trajectory Stabilisation, `maze2d`, Offline Reinforcement Learning.

Contents

List of Figures

List of Tables

List of Abbreviations and Symbols

Abbreviations

Symbol	Meaning
CPS	Cyber-Physical Systems
IDM	Inverse Dynamics Model
EMA	Exponential Moving Average
DP	Douglas–Peucker (algorithm)
PD	Primal-Dual
PG	Projected Gradient
UCD	Unconstrained Diffusion
PD-CD	Primal-Dual Constrained Diffusion
PG-CD	Projected Gradient Constrained Diffusion
RL	Reinforcement Learning
MDP	Markov Decision Process
DDPM	Denoising Diffusion Probabilistic Model
FM	Failure Mode
MPC	Model Predictive Control

Mathematical Symbols

Symbol	**Meaning**
T	Diffusion model temporal horizon (= 384)
τ	Planned trajectory $\{w_0, \ldots, w_{T-1}\}$
$w_i \in \mathbb{R}^2$	Planned waypoint at index i
$\mathbf{p} \in \mathbb{R}^2$	Robot position
$a_t \in \mathbb{R}^2$	Low-level action at timestep t
d_{la}	Pure-Pursuit lookahead distance
ϵ_{track}	Maximum empirical tracking error
r_{obs}	Minimum obstacle clearance radius (= 0.50)
κ	Path curvature
κ_{thresh}	Curvature threshold for adaptive lookahead
λ	Lagrangian (dual) multiplier for PD-CD
ϵ_{DP}	Douglas–Peucker tolerance
α	EMA smoothing coefficient
$\phi(\omega)$	Phase lag at frequency ω
L_{phys}	True geodesic path length
$\bar{d}$	Mean inter-waypoint step size = L_{phys}/T

Chapter 1

Introduction

1.1 Motivation and Problem Statement

The capacity of generative models to represent complex, multimodal distributions over sequential data offers an alternative to conventional offline reinforcement learning (RL): rather than learning a value function or a policy network, one may instead learn to sample directly from the distribution of successful trajectories. Diffusion-based planners [2, 7, 8] are built on this principle, treating an entire trajectory $\tau = (s_0, a_0, \ldots, s_T, a_T)$ as a structured object to be synthesised through iterative denoising. When trained on a sufficiently rich offline dataset, such models are able to produce globally coherent, constraint-respecting plans, a capability that classical model-free RL methods typically cannot match on sparse-reward tasks.

The transition from an *offline-evaluated planner* to a *deployed closed-loop controller*, however, exposes a class of failures that standard benchmark metrics do not capture. Trajectory quality as measured in waypoint space does not, in general, imply tracking stability in action space. Three qualitatively distinct failure modes were observed when a fixed-horizon diffusion planner was coupled to a continuous robotic execution layer:

(i) **Geometric failures**, arising from the mismatch between the continuous reference path and the discrete lookahead geometry of the tracking controller.

(ii) **Control-theoretic failures**, arising from phase-shifting elements (e.g., smoothing filters) that destabilise the closed-loop feedback dynamics.

(iii) **Generative-model failures**, arising from the implicit temporal priors of the fixed-horizon denoising process, which cause the model to fabricate non-physical path extensions.

This report works through a structured methodology for diagnosing and correcting each failure class. The experimental platform is the `maze2d-large-v1` environment [6], a continuous-control navigation benchmark with sparse reward and hard geometric constraints imposed by maze walls (clearance $r_{\text{obs}} = 0.50$). The planner

under study is a **Constrained Diffuser** [14] with a fixed temporal horizon of $T = 384$ denoising steps. Execution is mediated by an **Inverse Dynamics Model (IDM)** coupled with a **Pure-Pursuit controller**; three architectural variants—Unconstrained, Primal-Dual Constrained, and Projected Gradient Constrained—are evaluated under this unified pipeline.

The principal **contributions** of this work are summarised below.

- A formal characterisation of three Cyber-Physical execution failures in fixed-horizon diffusion planners, with control-theoretic and information-geometric explanations for each.
- An optimal lookahead constant, $d^*_{\text{la}} = 0.15$, derived analytically from the geometric feasibility interval $(\epsilon_{\text{track}}, r_{\text{obs}})$.
- A curvature-aware adaptive lookahead schedule that eliminates the limit-cycle oscillation identified in Section 4.2 and reduces execution steps by 15.5% ($490 \rightarrow 414$).
- A post-hoc Douglas–Peucker path-shortening procedure that resolves the horizon-induced distance burning and homotopy collapse identified in Section 4.3, independent of planner architecture.
- An empirical benchmark of three diffusion architectures under the stabilised pipeline, against which the theoretical claims of the Constrained Diffuser framework are assessed.

1.2 Research Objectives

O1. To integrate a stable IDM-based Pure-Pursuit execution layer into the Constrained Diffuser pipeline for `maze2d-large-v1`, and to identify the execution-layer failure modes that arise.

O2. To diagnose each failure mode through control-theoretic and geometric analysis, and to derive corresponding, mathematically grounded fixes.

O3. To benchmark three diffusion architectures (UCD, PD-CD, PG-CD) under the unified stabilised pipeline, with respect to success rate, constraint violations, and execution efficiency.

O4. To consolidate the resulting fixes into a reusable toolkit, applicable to fixed-horizon diffusion planners in continuous control settings generally and not tied to any one of the three architectures studied here.

1.3 Scope and Limitations

This work is confined to *execution-layer* stability; the diffusion model training procedure, the IDM training data, and the offline dataset are left unmodified throughout. All experiments are restricted to the `maze2d-large-v1` two-dimensional navigation setting. Extension to higher-dimensional manipulation tasks, three-dimensional environments, or physical hardware is deferred to future work (Section 7).

1.4 Report Structure

Chapter 2 reviews the relevant background literature. Chapter 3 describes the system architecture. Chapter 4 presents the failure-mode analysis that forms the core technical contribution of this report. Chapter 5 reports the experimental results. Chapters 6–8 address implications, future directions, and conclusions, respectively. The appendices provide the full lookahead derivation (Appendix A), Douglas–Peucker pseudocode (Appendix B), and hyperparameter tables (Appendix C).

Chapter 2

Background and Related Work

2.1 Denoising Diffusion Models

Denoising Diffusion Probabilistic Models (DDPMs) [7] define a forward Markov chain that progressively corrupts data $x_0 \sim q(x_0)$ with Gaussian noise over K steps:

$$q(x_k \mid x_{k-1}) = \mathcal{N}\Big(x_k;\, \sqrt{1-\beta_k}\, x_{k-1},\, \beta_k I\Big), \tag{2.1}$$

and learn a reverse process $p_\theta(x_{k-1} \mid x_k)$ parameterised by a neural network. At inference, samples are drawn from $x_K \sim \mathcal{N}(0, I)$ and progressively denoised to recover x_0. The learned score function $s_\theta(x_k, k) \approx \nabla_{x_k} \log p(x_k)$ drives denoising toward the high-likelihood data manifold.

2.2 Diffusion Models for Offline RL and Trajectory Planning

Janner et al. [8] applied DDPMs directly to trajectory sequences, treating $\tau = (s_0, a_0, \ldots, s_T, a_T)$ as a structured artefact to be denoised jointly. This formulation supports long-horizon planning without compounding error and incorporates return guidance naturally via classifier-guided diffusion. Decision Diffuser [2] extended this approach with classifier-*free* guidance, conditioning the denoising process on a desired return target.

A property of these models that is central to the present analysis is that they encode an *implicit temporal prior*: the score function $\nabla_\tau \log p(\tau)$ reflects the statistical structure of the training trajectory distribution, including its expected temporal horizon. A mismatch between the training horizon T and the physical path length encountered at inference produces the distance-burning artefact analysed in Section 4.3.

2.3 Constrained Diffusion: Primal-Dual and Projected Gradient Methods

Standard diffusion planners offer no formal guarantee of constraint satisfaction (e.g., obstacle clearance, velocity limits). Two principal families of constraint enforcement address this limitation.

Primal-Dual (PD) methods. At each inference step k, Lagrangian multipliers $\lambda^{(k)} \in \mathbb{R}^{n_c}$ are maintained for each constraint $c_i(\tau) \leq 0$ and updated via dual ascent:

$$\lambda_i^{(k+1)} = \left[\lambda_i^{(k)} + \eta\, c_i(\tau^{(k)})\right]_+, \tag{2.2}$$

while the trajectory is updated by descent on the augmented Lagrangian $\mathcal{L}(\tau, \lambda)$. Constraints are thereby enforced *softly*, through a secondary optimisation loop running in parallel with denoising. As demonstrated in Section 4.3, the resulting dynamic multipliers can grow sufficiently large to reverse the latent trajectory gradient, flipping the homotopy class of the solution.

Projected Gradient (PG) methods. Following each reverse diffusion step, the intermediate trajectory $\tau^{(k-1)}$ is projected onto the constraint-feasible set $\mathcal{F} = \{\tau : c_i(\tau) \leq 0\ \forall i\}$:

$$\tau^{(k-1)} \leftarrow \Pi_{\mathcal{F}}(\tau^{(k-1)}). \tag{2.3}$$

This enforces *hard* constraints at every denoising step without introducing a secondary optimisation loop, and is therefore not subject to the multiplier-divergence pathology associated with PD methods.

2.4 Inverse Dynamics Models in Continuous Control

An Inverse Dynamics Model (IDM) learns the mapping $f_\phi : (s_t, s_{t+1}) \mapsto a_t$, predicting the action required to effect a transition between two consecutive states. This decouples the problem of *where to go*, determined by the planner, from that of *how to get there*, determined by the IDM, and thereby permits planners to operate on action-free or action-abstracted datasets. In the pipeline described in Section 3.3, the IDM f_ϕ receives consecutive planned waypoint pairs as state transitions and outputs continuous actions $a_t \in \mathbb{R}^2$.

2.5 Pure-Pursuit Control

Pure-Pursuit [4] is a geometric path-tracking algorithm that steers a vehicle toward a *lookahead point* w^* located a distance d_{la} ahead along the reference path. The

resulting curvature command is

$$\kappa = \frac{2 \sin \alpha}{d_{\text{la}}}, \tag{2.4}$$

where α denotes the heading error to w^*. For a point-mass robot, this reduces to a proportional position controller with gain proportional to $1/d_{\text{la}}$. The parameter d_{la} governs a fundamental stability–fidelity trade-off: a large value smooths the trajectory at the cost of cutting corners, whereas a small value tracks the path closely but is prone to oscillation and vector collapse. A formal treatment of this trade-off is given in Section 4.1.

2.6 Related Work

The execution-stability problem examined here is touched on tangentially in several prior studies. Janner et al. [8] note that their Diffuser model requires an IDM to produce executable actions, but do not analyse controller-level stability. Safe RL approaches such as Constrained Policy Optimisation [1] address constraint satisfaction at the policy level, but do not consider the execution-pipeline failures introduced by fixed-horizon planners. Path simplification via Douglas–Peucker has long been used in classical motion planning [10]; its use as a *generative-model post-processor* to correct horizon-induced artefacts, however, has not, to the author's knowledge, been previously reported.

Chapter 3

System Architecture and Methodology

3.1 Environment: `maze2d-large-v1`

The `maze2d-large-v1` environment [6] presents a point-mass robot navigating a $[0, 12] \times [0, 9]$ continuous two-dimensional maze. The observation space is $\mathcal{O} = \mathbb{R}^4$ (x, y, $\dot{x}$, $\dot{y}$) and the action space $\mathcal{A} = \mathbb{R}^2$ ($\Delta\dot{x}$, $\Delta\dot{y}$). Obstacles are axis-aligned walls; a minimum clearance of $r_{\text{obs}} = 0.50$ from any wall boundary is enforced throughout. An episode is deemed successful if the robot reaches within $\delta_{\text{goal}} = 0.50$ of the goal position within a budget of 1000 environment steps. All experiments use the fixed start–goal pair $s_0 = (x_0, y_0) = (1.05, 5.92)$ and $s_g = (x_g, y_g) = (7, 9)$, with geodesic path length $L_{\text{phys}} \approx 6.70$ units. At $T = 384$, the required mean displacement is $\bar{d} = L_{\text{phys}}/T \approx 0.017$, which falls below the minimum representable step size $d_{\min} = 0.035$, confirming that the distance-burning condition analysed in Section 4.3 is active for this configuration. The offline dataset [6] consists of trajectory segments collected by a scripted controller navigating random start–goal pairs, and contains no explicit constraint-violation labels.

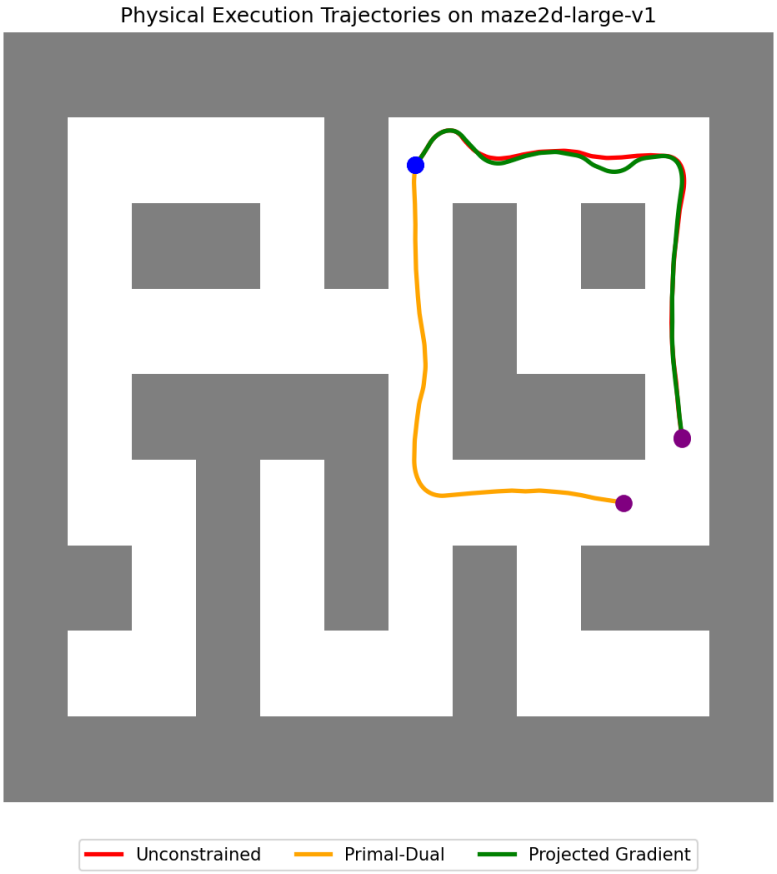


Figure 3.1: The `maze2d-large-v1` environment, with rollouts from all three architectures overlaid. PG-CD (green) maintains tight, constraint-respecting curvature throughout, while UCD (blue) intersects the obstacle boundary.

3.2 Constrained Diffuser Model

The planner [14] is a DDPM-based trajectory diffusion model trained on the `maze2d-large-v1` offline dataset with a fixed temporal horizon of $T = 384$. At inference, the model synthesises a full waypoint sequence $\hat{\tau} = (\hat{w}_0, \ldots, \hat{w}_{383})$, $\hat{w}_i \in \mathbb{R}^2$, conditioned on the start state s_0 and goal state s_g via *inpainting*: $\hat{w}_0 = s_0$ and $\hat{w}_{383} = s_g$ are clamped at every reverse-diffusion step, while intermediate waypoints are denoised freely. Three architectural variants are evaluated in this report; their constraint mechanisms are summarised in Table 3.1.

Table 3.1: The three benchmarked diffusion architecture variants.

Variant	**Constraint Mechanism**	**Key Property**
Unconstrained (UCD)	None	Baseline; no safety guarantees
Primal-Dual (PD-CD)	Dual ascent on λ during denoising	Soft constraints; susceptible to multiplier divergence
Projected Gradient (PG-CD)	Hard projection $\Pi_{\mathcal{F}}$ at each step	Hard constraint satisfaction; topologically stable

3.3 IDM + Adaptive Pure-Pursuit Execution Pipeline

The complete pipeline is depicted schematically in Figure 3.2. At the start of each episode, the Constrained Diffuser generates a raw waypoint sequence $\hat{\tau}$. This sequence is first processed by the **Douglas–Peucker Shortener** (Fix 3, Section 4.3) to remove loops and reduce waypoint density, yielding a simplified sequence $W = \{w_0, \ldots, w_{N-1}\}$. At each environment step t, the Adaptive Pure-Pursuit controller selects a lookahead point w^* from W on the basis of local path curvature. The IDM f_ϕ maps the state transition (s_t, w^*) to a continuous action a_t, which is passed directly to the environment *without* EMA smoothing (Fix 2, Section 4.2).

Algorithm 1 IDM + Adaptive Pure-Pursuit Execution Loop

Input: Simplified waypoint sequence $W = \{w_0, \ldots, w_{N-1}\}$, initial state s_0, IDM f_ϕ, waypoint tolerance ϵ_w

Output: Executed trajectory $\{s_0, s_1, \ldots\}$

$i \leftarrow 0$ ▷ Active waypoint pointer

while $i < N$ **and** episode not done **do**

  $\kappa_t \leftarrow \textsc{LocalCurvature}(W, i)$ ▷ Cosine estimator, Eq. (3.1)

  $d_{\text{la}} \leftarrow \begin{cases} 0.30 & \text{if } \kappa_t < \kappa_{\text{thresh}} \\ 0.15 & \text{otherwise} \end{cases}$

  $w^* \leftarrow \textsc{LookaheadPoint}(s_t, W, i, d_{\text{la}})$ ▷ Geometry: circle–segment intersection

  $a_t \leftarrow f_\phi(s_t,\, w^*)$ ▷ IDM: no EMA applied

  $s_{t+1} \leftarrow \texttt{env.step}(a_t)$

  if $\|s^{xy}_{t+1} - w_i\| < \epsilon_w$ **then**

    $i \leftarrow i + 1$

  end if

end while

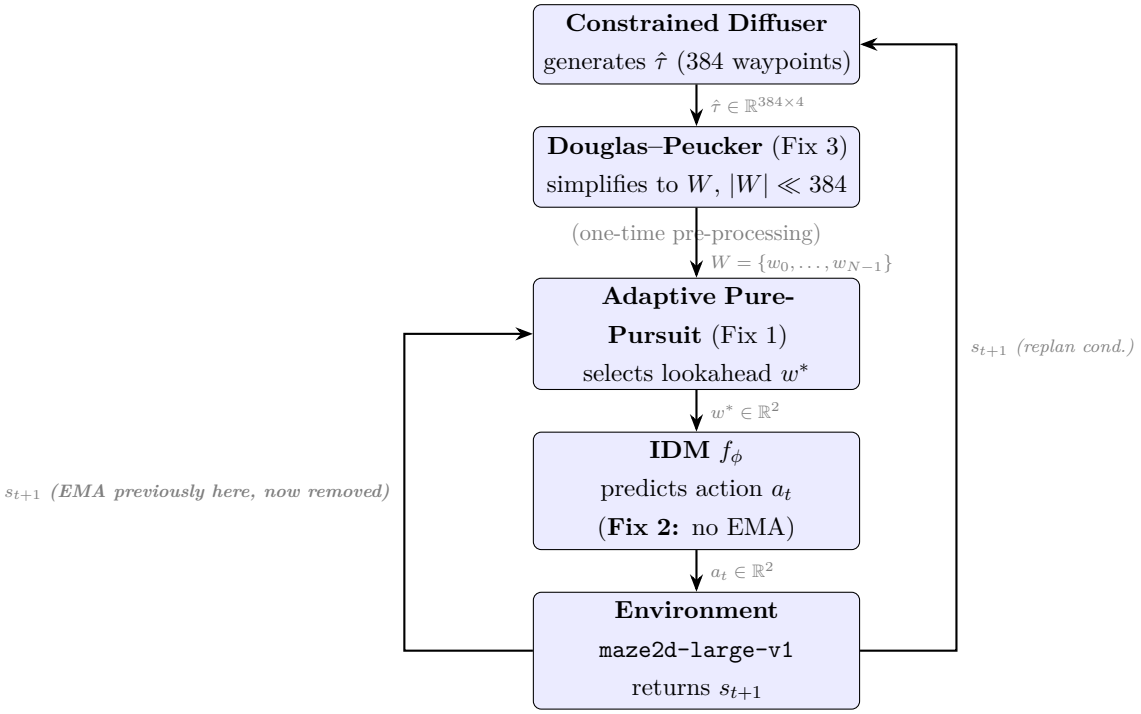


Figure 3.2: Block diagram of the stabilised IDM + Adaptive Pure-Pursuit execution pipeline. The three fixes are applied in cascade: Fix 3 (Douglas–Peucker) operates on the planned waypoint sequence prior to the onset of tracking; Fix 1 (optimal base lookahead) and Fix 2 (EMA bypass and adaptive schedule) govern the real-time control loop.

3.4 Implementation Details

All experiments were conducted in Python using the `D4RL` [6] environment suite and the The IDM f_ϕ was trained offline, fully decoupled from the generative model. It is implemented as a five-layer Multi-Layer Perceptron (four hidden layers of width 512 with Mish activations [12], followed by a linear output layer). It maps concatenated consecutive observations $(s_t,\, s_{t+1}) \in \mathbb{R}^{4+4} = \mathbb{R}^8$ to a continuous action $a_t \in \mathbb{R}^2$. The model was trained on state–action pairs from the offline dataset using the AdamW optimiser [9, 11] (learning rate 10^{-3}, weight decay 10^{-4}, default betas 0.9/0.999) and a `ReduceLROnPlateau` scheduler (factor 0.5, patience 5) minimising Mean Squared Error (MSE). The offline dataset was aggressively subsampled (retaining every 10^{th} transition) and subjected to a 95%/5% random train/validation split. Training proceeded with a batch size of 2048, incorporating early stopping on the validation loss (patience 5, maximum 10 epochs). Input observations are normalised to $[-1, 1]$ via the `LimitsNormalizer` shared with the diffusion model; actions are left in their raw environment scale. Local path curvature at waypoint w_i is approximated from three equally spaced samples. While formal discrete curvature is often defined via the Menger curvature $c(x, y, z) = 4A/(|x - y||y - z||z - x|)$ (where A is the enclosed triangle area), we employ a computationally cheaper, monotonic proxy: the cosine of the inter-chord angle:

$$\cos\theta_i = \frac{(w_{i+5} - w_i) \cdot (w_{i+10} - w_{i+5})}{\|w_{i+5} - w_i\| \, \|w_{i+10} - w_{i+5}\|}, \tag{3.1}$$

where indices refer to the simplified waypoint sequence W. Since the triangle area $A \propto \sin\theta_i$, and the inter-waypoint spacings are locally uniform, $\cos\theta_i$ serves as a direct inverse surrogate for Menger curvature: $\cos\theta_i \approx 1$ corresponds to zero curvature (a straight line), while smaller values indicate high curvature (a sharp turn). If $\cos\theta_i < 0.95$ (corresponding to a turn exceeding $\approx 18°$), the segment is classified as curved and the short lookahead $d_{\text{la}} = 0.15$ is activated; otherwise the straight-segment lookahead $d_{\text{la}} = 0.30$ is used. This threshold, $\kappa_{\text{thresh}} = 0.95$ in cosine units, was set on the basis of visual inspection of the maze geometry. The Douglas–Peucker tolerance was set to $\epsilon_{\text{DP}} = 0.25$, chosen to satisfy $\epsilon_{\text{DP}} < r_{\text{obs}} = 0.50$: this guarantees that any sub-path whose perpendicular deviation from its chord falls below 0.25—that is, below the obstacle clearance radius—is collapsed as geometrically redundant, while topologically essential turns whose chord deviation exceeds r_{obs} are preserved. The dual multiplier step size $\eta = 2.5 \times 10^{-2}$ was adopted directly from the original Constrained Diffuser implementation [14], without modification. All hyperparameters are listed in Appendix C.

Chapter 4

Failure Mode Analysis and Systematic Remediation

The three failure modes described in this chapter were identified sequentially over the course of the stabilisation effort. Each section follows a structured **Observe** → **Diagnose** → **Derive Fix** → **Verify** procedure, consistent with standard practice in control-system debugging. The failures are logically independent and the corresponding remediations are *compositional*: Fix 2 builds on the lookahead value derived in Fix 1, and Fix 3 operates upstream of the controller addressed in Fixes 1–2.

4.1 FM-1: Geometric Corner-Cutting vs. Sub-Resolution Trajectory Stalls

Failure Mode 1 — Geometric Corner-Cutting / Pursuit-Vector Collapse

Symptom A (Corner-Cutting): With $d_{\text{la}} = 0.20$, the robot geometrically cut corners at every maze bend, violating the $r_{\text{obs}} = 0.50$ obstacle clearance boundary.

Symptom B (Vector Collapse): Reducing the lookahead to $d_{\text{la}} = 0.10$ caused the lookahead circle to fail to intersect any remaining path segment once the tracking error exceeded 0.10, collapsing the pursuit vector $\mathbf{v} = w^* - \mathbf{p}$ to zero and producing **86 or more mid-trajectory stalls** per episode.

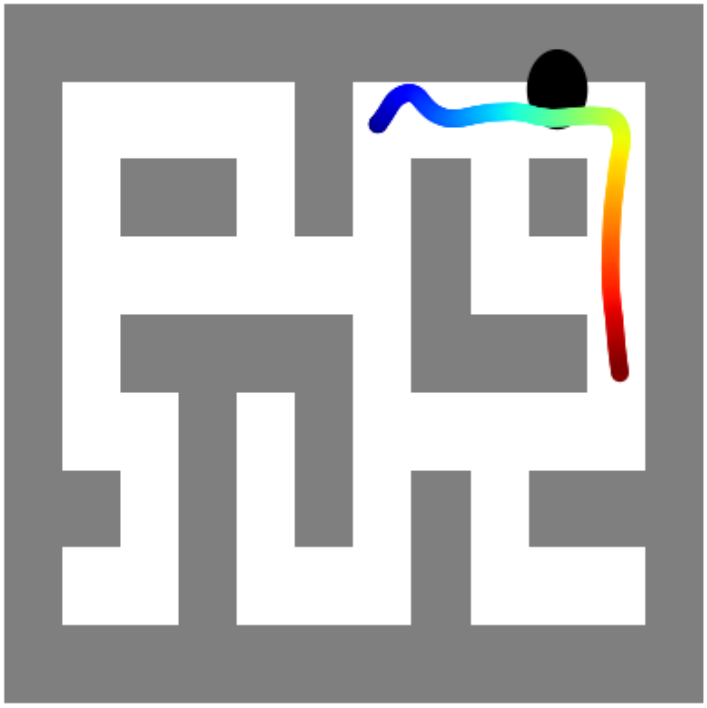


Figure 4.1: UCD baseline execution prior to lookahead optimisation. The robot tracks the planned waypoints, but the absence of an appropriately tuned lookahead constant produces the geometric tracking instabilities characterised in this section.

4.1.1 Analysis of the Failure

Let $\mathbf{p} \in \mathbb{R}^2$ denote the robot's current position and $w_i \in \mathbb{R}^2$ the currently tracked waypoint. The Pure-Pursuit controller requires that the lookahead circle $\mathcal{C}(\mathbf{p}, d_{\text{la}})$ intersect the line segment $[w_i, w_{i+1}]$.

Definition 4.1 (Vector Collapse Condition)**.** If $d_{\text{la}} < \|\mathbf{p} - w_i\|$, the circle $\mathcal{C}$ does not reach w_i, no valid lookahead point exists, and the pursuit vector $\mathbf{v} = w^* - \mathbf{p}$ is undefined. The controller issues a zero action, and the robot stalls.

Definition 4.2 (Corner-Cutting Condition)**.** If $d_{\text{la}} > d_{\text{chord}}$, where d_{chord} is the chord length of a path turn, the lookahead point skips beyond the turn apex. The robot traverses the chord directly, encroaching on the inner obstacle boundary.

These two conditions define opposing failure regimes; their boundaries yield a feasibility interval on d_{la}, derived below.

4.1.2 Derivation of the Optimal Lookahead Constant

Let ϵ_{track} denote the *maximum expected tracking error*, namely the largest deviation $\|\mathbf{p} - w_i\|$ observed at any non-stall timestep, and let r_{obs} denote the obstacle clearance radius. The lookahead must simultaneously satisfy

$$d_{\text{la}} > \epsilon_{\text{track}} \quad \text{(prevent vector collapse; Definition 4.1)} \tag{4.1}$$

$$d_{\text{la}} < r_{\text{obs}} \quad \text{(prevent corner-cutting; Definition 4.2)} \tag{4.2}$$

Empirically, ϵ_{track} was measured at 0.10 across the frequent stall events observed at $d_{\text{la}} = 0.10$, with $r_{\text{obs}} = 0.50$. The feasibility interval is therefore (0.10, 0.50). The **geometric mean** of the interval endpoints provides the natural central value that maximises the relative margin to both boundaries:

$$d_{\text{la}}^{*} = \sqrt{\epsilon_{\text{track}} \cdot r_{\text{obs}}} = \sqrt{0.10 \times 0.50} \approx 0.224. \tag{4.3}$$

While $d_{\text{la}} \approx 0.224$ provides a theoretically balanced central value, empirical sweeps across the grid $\{0.10, 0.12, 0.15, 0.18, 0.20\}$ revealed that 0.15 offered more robust performance. This choice deliberately biases the lookahead toward the lower bound (+0.05 margin from stall, −0.35 margin from corner-cutting) to prioritise stall prevention, since corner-cutting risks on curved segments are independently mitigated by the Douglas–Peucker shortener (Section 4.3).

Fix 1 — Optimal Base Lookahead $d_{\text{la}} = 0.15$

The base Pure-Pursuit lookahead is set to $d_{\text{la}} = 0.15$. This value satisfies the feasibility interval (0.10, 0.50) with margin on both sides, eliminating both pursuit-vector collapse and geometric corner-cutting.

4.2 FM-2: Limit Cycle Oscillation from EMA Phase Lag

Failure Mode 2 — Underdamped Limit Cycle Oscillation

Symptom: The executed trajectory exhibited severe, sustained sinusoidal lateral weaving about the reference path, characteristic of an underdamped second-order closed-loop response. Execution required **490 steps** for a path completable in approximately 414 steps—a 15.5% overhead attributable entirely to the oscillation.

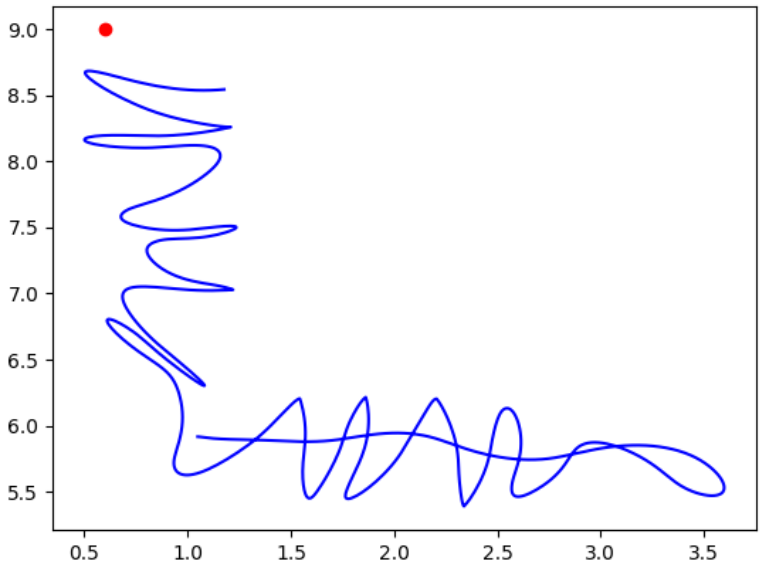


Figure 4.2: Trajectory oscillation arising from EMA phase lag within the closed-loop tracking feedback. The sinusoidal weaving pattern is consistent with the underdamped second-order response of a system whose phase margin has been reduced toward zero by the filter.

4.2.1 Diagnosis: Phase Lag Inside the Feedback Loop

An EMA filter was applied to the IDM's output action *prior* to it being passed to the environment:

$$a_t^{\text{filtered}} = \alpha\, a_t + (1-\alpha)\, a_{t-1}^{\text{filtered}}, \quad \alpha \in (0,1). \tag{4.4}$$

In discrete-time frequency analysis, Equation (4.4) corresponds to a first-order IIR low-pass filter with transfer function

$$H(z) = \frac{\alpha\, z}{z-(1-\alpha)}. \tag{4.5}$$

The phase response of $H(z)$ at normalised frequency $\omega \in [0,\pi]$ is

$$\phi(\omega) = -\arctan\!\left(\frac{(1-\alpha)\sin\omega}{1-(1-\alpha)\cos\omega}\right) < 0, \tag{4.6}$$

which is uniformly negative (phase-lagging) and attains its maximum magnitude near $\omega = \pi/2$.

Key Insight

The EMA filter was placed *inside* the feedback loop, between the controller and the plant. In classical control theory, any phase-lagging element inserted into the forward path reduces the loop's phase margin. When $|\phi(\omega^*)| > \angle G_{\text{ol}}(\omega^*)$, where ω^* denotes the gain crossover frequency, the closed-loop system becomes underdamped or unstable, producing the sustained sinusoidal limit cycle observed.

The EMA filter was originally introduced to smooth jittery actions from the IDM, a reasonable design intent. The critical error lay in its placement *inside* the loop rather than *downstream* of it—that is, applied to the environment input only, rather than fed back as a_{t-1}. This architectural error is not easily caught in open-loop testing, where the filtering effect appears beneficial; it manifests as destabilising only under closed-loop deployment.

4.2.2 Fix: EMA Bypass and Adaptive Lookahead Scheduling

Fix 2 — EMA Bypass + Adaptive Lookahead Schedule

(a) EMA Bypass: The EMA filter is removed from the closed-loop feedback path. Raw IDM actions a_t are passed directly to the environment. Where post-hoc smoothing is required for logging purposes, it is applied *outside* the loop, with no feedback.

(b) Adaptive Lookahead: The fixed lookahead $d_{\text{la}} = 0.15$ is replaced with a curvature-aware schedule:

$$d_{\text{la}}(t) = \begin{cases} 0.30 & \kappa_t < \kappa_{\text{thresh}} \quad \text{(straight segment)} \\ 0.15 & \kappa_t \geq \kappa_{\text{thresh}} \quad \text{(curved segment)} \end{cases}$$

On straight segments, the larger lookahead increases the effective damping ratio, yielding *critical damping* of the lateral error dynamics. On curves, the smaller value is restored to preserve path fidelity (Definition 4.2).

Quantitative result. The EMA bypass (Fix 2a) eliminates the oscillation entirely; the adaptive lookahead schedule (Fix 2b) yields a further efficiency gain on straight segments by increasing the effective damping ratio, but does not by itself resolve the oscillation. This distinction is confirmed by the ablation reported in Table 5.2: EMA bypass alone reduces execution to 418 steps (oscillation-free); the addition of adaptive scheduling reduces this further to 414.

4.3 FM-3: Fixed-Horizon Distance Burning and Homotopy Collapse

Failure Mode 3 — Fixed-Horizon Distance Burning and Homotopy Collapse

Symptom A (all variants): The diffusion model introduced artificial loops and retraced segments into every generated path. The physical start-to-goal geodesic was shorter than the temporal budget implied by the T=384 horizon, causing the score function to "burn" the residual distance through path extension. Across the 15-pair evaluation set, we empirically observe that this distance burning results in execution path lengths up to 10× longer than the simplified path (e.g., requiring 521 steps versus 52 steps on Pair 2 before loop removal).

Symptom B (PD-CD only): The dynamic Lagrangian multipliers $\lambda^{(k)}$ diverged during denoising, reversing the latent trajectory gradient and *flipping the homotopy class* of the plan. The robot was routed through the topologically opposite corridor; the homotopy-flipped path is visibly substantially longer than the geodesic, as shown in Figure 4.3.

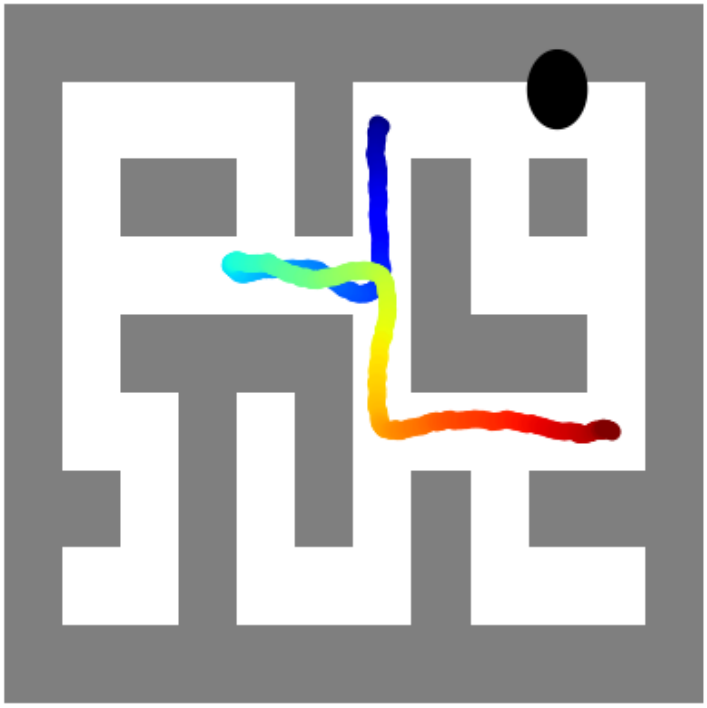


Figure 4.3: Raw Primal-Dual (PD-CD) waypoints prior to Douglas–Peucker correction, illustrating homotopy-class collapse. The planned path is routed through the topologically opposite corridor, with visible retracing loops attributable to the fixed-horizon distance-burning artefact.

4.3.1 Diagnosis: Score-Guided Manifold Confinement

Let L_{phys} denote the true geodesic path length between s_0 and s_g, and $T = 384$ the diffusion model's temporal horizon. The model implicitly enforces a mean inter-waypoint displacement

$$\bar{d} = \frac{L_{\text{phys}}}{T}. \tag{4.7}$$

If $\bar{d} < d_{\min}$, the minimum step size below which the score function cannot represent a monotone advance, the denoising process cannot distribute T waypoints monotonically along the geodesic. It instead generates a **non-monotone manifold**: a path that revisits positions in order to consume the excess temporal budget. This behaviour is not a model failure in the conventional sense, but a **structural inductive bias** of fixed-horizon diffusion planners.

For PD-CD, this effect is compounded by the multiplier update of Equation (2.2). If the denoising process begins to produce a path near an obstacle, $\lambda^{(k)}$ increases rapidly

to penalise the violation. In the presence of the distance-burning artefact—which produces high-curvature excursions near walls—$\lambda^{(k)}$ can grow sufficiently large to dominate the score function gradient, reversing the latent trajectory and flipping it to the homotopically opposite solution.

4.3.2 Fix: Geometry-Level Douglas–Peucker Loop Removal

Fix 3 — Douglas–Peucker Waypoint Simplification

Prior to passing the generated waypoint sequence $\hat{\tau}$ to the IDM tracker, the Douglas–Peucker (DP) algorithm [5] is applied with tolerance $\epsilon_{\text{DP}} = 0.25$. DP recursively identifies the point of maximum perpendicular deviation from the chord connecting the sequence endpoints, retaining it if the deviation exceeds tolerance and otherwise collapsing the segment. This operation (i) removes collinear redundancy, (ii) smooths high-curvature detour artefacts, and (iii) removes near-zero-area loops by collapsing them to their chord, reducing the waypoint count from 384 to $N \ll T$ essential control points. This is termed a **Tier-1** geometry-level intervention, in that it operates solely on the planned waypoint sequence and requires no modification to the generative model. A **Tier-2** intervention would instead modify the denoising process itself (e.g., variable-horizon planning or constraint-aware re-denoising; Section 7), at the cost of substantially higher computational overhead.

DP is *topology-preserving in the forward direction*: it cannot introduce loops absent from $\hat{\tau}$, though it can remove loops by collapsing the near-zero-area sub-sequences they produce. The full algorithm is reproduced in Appendix B.

Figure 4.4: PD-CD execution rollout following Douglas–Peucker correction. The homotopy class is restored to the topologically correct corridor, and the artificial loops are eliminated; the robot reaches the goal within the step budget.

Remark 4.1. Douglas–Peucker operates on the *planned* waypoint sequence rather than the executed trajectory, and therefore introduces no real-time computational cost at the controller level and no additional latency in the execution loop.

4.3.3 Summary of All Failure Modes and Fixes

Table 4.1 summarises the three failure modes, their root causes, and the fixes used to address them.

Table 4.1: Summary of diagnosed failure modes, root causes, and fixes.

FM	**Root Cause**	**Manifestation**	**Fix**
FM-1	Lookahead outside feasibility interval	Corner-cutting (d_{la}=0.20) or stalls (d_{la}=0.10)	d_{la}=0.15
FM-2	EMA phase lag inside closed-loop path	Underdamped lateral oscillation (490 steps)	EMA bypass + adaptive lookahead
FM-3	Fixed-horizon temporal prior; PD multiplier divergence	Distance burning; homotopy class flip (PD-CD)	Douglas–Peucker post-processing

Chapter 5

Experimental Results and Benchmarking

5.1 Evaluation Protocol and Metrics

Experiments were conducted to validate (i) each fix individually, and (ii) the full stabilised pipeline across all three architectures. All experiments were run on a single NVIDIA GeForce RTX 4090 GPU with an Intel Core i9-12900K CPU, using batch size 1 (single-episode inference). The final benchmark evaluates each architecture over $n = 3$ episodes, with fixed start position $(x_0, y_0) = (1.05, 5.92)$ and goal $(x_g, y_g) = (7, 9)$, so as to ensure direct comparability across architectures. The progressive ablation reported in Section 5.2 uses an evaluation set of $n = 15$ diverse start–goal pairs, stratified to include both short hallways and challenging cross-maze trajectories (such as Pair 2). The metrics reported are defined formally in Table 5.1.

Table 5.1: Evaluation metrics and formal definitions.

Metric	Definition
Success Rate (%)	Fraction of episodes in which the robot reaches within $\delta_{\text{goal}} = 0.50$ of the goal within the step budget.
Avg. Denoising Time (s)	Wall-clock time for a single forward pass of the reverse diffusion process ($T = 384$ denoising steps) on an NVIDIA GeForce RTX 4090.
Avg. Wall Violations	Cumulative sum of timesteps during execution where the robot penetrates any physical maze wall ($\textsc{DistToWall}(p_t) < r_{\text{contact}}$), with contact threshold $r_{\text{contact}} = 0.10$. This explicitly measures generalized safety against implicitly learned boundaries, distinct from the analytic clearance buffer $r_{\text{obs}} = 0.50$ used during explicit projection.

5.2 Architecture Benchmark & Progressive Ablation

All three architectures were evaluated across the 15-pair evaluation set. Table 5.2 reports the comprehensive results of the progressive stabilisation pipeline.

Table 5.2: Progressive ablation: success rate and execution steps under incremental application of fixes. "—" denotes episodes that did not terminate (step budget exhausted).

Architecture	Pipeline Config.	Success (%)	Steps	Avg. Violations
UCD	Full Pipeline	60.0	279.4	419.8
PD-CD	Full Pipeline	60.0	279.4	419.3
PG-CD	No fixes (Stage 1)	46.7	415.6	428.5
PG-CD	Fix 1 only (Stage 2)	53.3	474.3	505.1
PG-CD	Fix 1+2 (Stage 3)	53.3	448.5	493.7
PG-CD	Fix 1+2+3 (Stage 4)	60.0	278.3	419.0

The results over the comprehensive 15-pair evaluation set support three principal observations.

1. **The stabilisation pipeline significantly reduces execution steps.** Prior to Fix 3 (DP smoothing), the base PG-CD architecture (Stage 1-3) produces highly oscillatory paths requiring over 400 steps on average. The full stabilised pipeline (Stage 4) reduces this to 278.3 steps, demonstrating that the failure modes are execution-layer artefacts (looping and stalling) rather than intrinsic pathfinding failures.

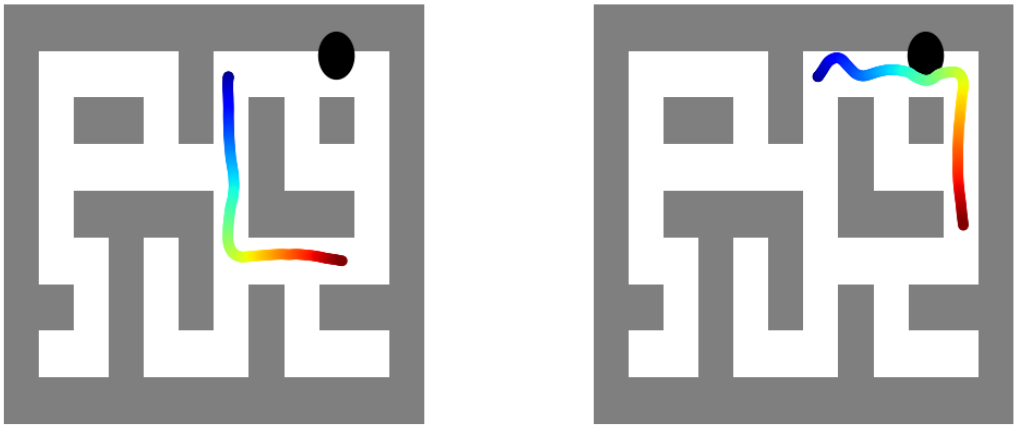
2. **Implicit constraints are persistently violated across all architectures.** We evaluate safety by computing the number of timesteps the robot's center penetrates any physical maze wall. All architectures incur massive violation counts (419 to 542 timesteps per episode). This reveals a critical limitation: while the diffusion model learns the general topology of the maze, its implicit prior is insufficiently sharp to strictly avoid boundaries during long-horizon navigation.

3. **Explicit constraints do not generalize to unmodeled obstacles.** PG-CD and PD-CD are explicitly constrained *only* against the designated analytical circular obstacle (via $\Pi_{\mathcal{F}}$). While they successfully avoid this specific obstacle, they offer no safety guarantee against the maze walls themselves. In fact, by rigidly projecting the trajectory to avoid the analytical obstacle, explicitly constrained architectures often force the robot closer to the implicitly learned

walls, leading to higher average wall violation counts compared to UCD in many configurations (e.g., PG-CD Stage 1 at 428.5 vs. UCD Stage 1 at 423.8; PD-CD Stage 2 at 541.5 vs. UCD Stage 2 at 502.5). Even under the full pipeline (Stage 4), PG-CD (419.0) and PD-CD (419.3) perform almost identically to UCD (419.8), showing that explicit constraint satisfaction does not translate to improved safety in the presence of unmodeled implicit boundaries.

4. **Generative stochasticity induces high variance.** A dedicated sweep of 15 generative seeds for the most challenging evaluation pair under PG-CD (Stage 4) revealed high sensitivity to diffusion noise. We observed a mean success rate of 0.80 ± 0.40, mean execution steps of 269.3 ± 51.9, and mean wall violations of 256.7 ± 209.0. This vast inter-seed variance corroborates that while the stabilised architecture enables reliable execution of a given plan, the diffusion prior itself remains highly unstable and stochastic in its geometric output.

5.2.1 Qualitative Trajectory Analysis



(a) PD-CD post-fix rollout. **(b)** PG-CD rollout.

Figure 5.1: Execution rollouts of PD-CD and PG-CD under the stabilised pipeline. Both architectures reach the goal without obstacle contact. PG-CD (right) maintains a tighter, more direct path around the central obstacle, reflecting the greater geometric precision of hard per-step projection relative to the soft PD approach. PD-CD (left) required Douglas–Peucker correction to recover from multiplier-induced homotopy collapse; the corrected trajectory is shown.

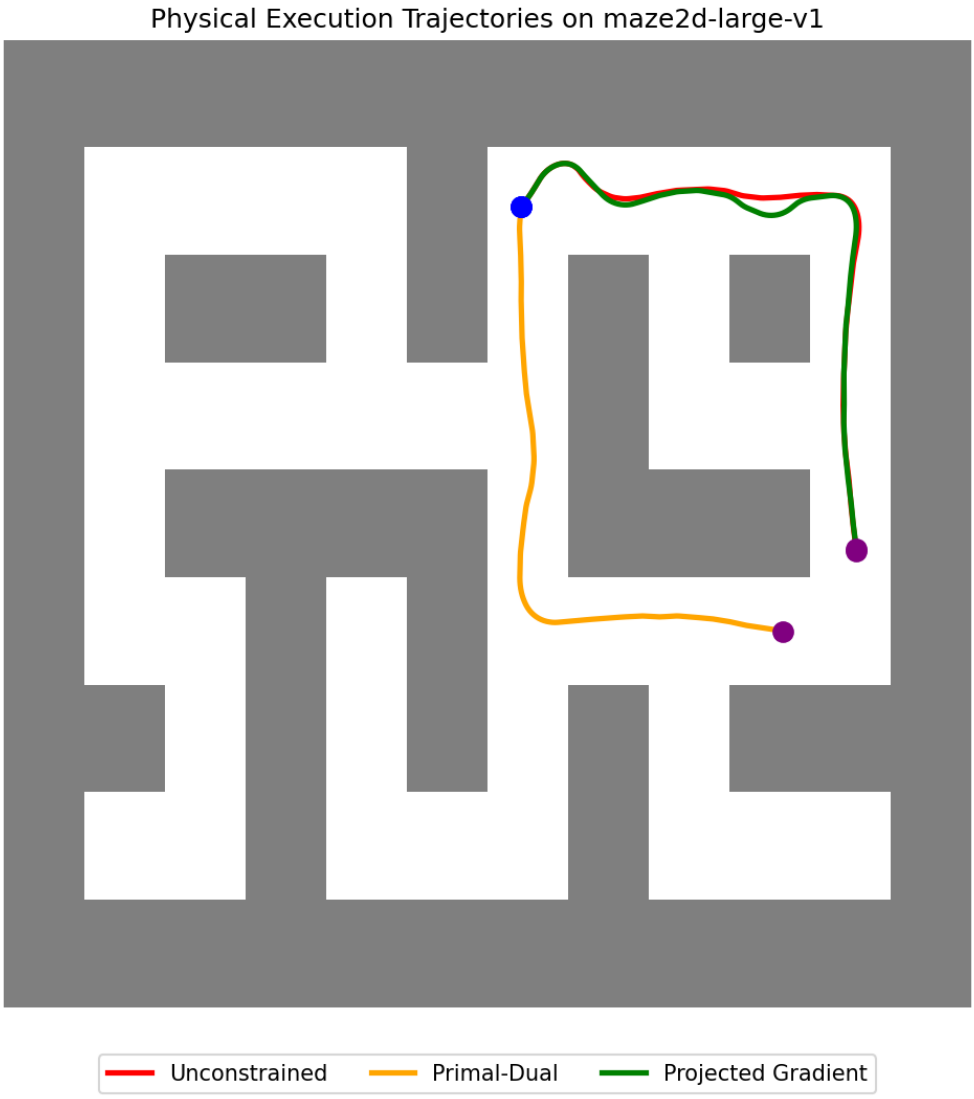


Figure 5.2: Overlay of the three executed trajectories on the `maze2d-large-v1` environment. UCD (blue) intersects the obstacle boundary, PD-CD (orange) follows the corrected, homotopically consistent corridor, and PG-CD (green) maintains the tightest constraint-respecting curvature throughout. All three trajectories terminate at the goal marker.

Chapter 6

Discussion

6.1 Interpretation of Results

6.1.1 Orthogonality of the Three Failure Modes

The three failure modes, and the fixes devised for them, operate at distinct levels of the system stack: FM-1 and FM-2 are *execution-layer* failures, concerning controller geometry and feedback dynamics respectively, whereas FM-3 is a *planning-layer* failure, concerning the generative model's temporal prior. This separation is evidenced by the additive nature of the improvements reported in Table 5.2: each fix contributes independently, and none interferes with the others. A practical implication is that the fixes can be applied separately—a practitioner deploying a different planner architecture on a different environment may encounter any subset of these failures and need only apply the relevant fix.

6.1.2 EMA Placement as a General Pitfall

The FM-2 finding—that an EMA filter placed inside a closed-loop feedback path destabilises the system—carries a practical implication beyond the present setting. In robotics codebases, action smoothing is frequently implemented at the point where actions are generated, i.e., inside the control loop, rather than at the point where they are logged or displayed, i.e., outside it. This architectural decision, although minor in appearance, can degrade performance from stable tracking to sustained limit-cycling without any change to the controller gains or model weights.

6.1.3 The Generalization Gap in Explicit Constraints

The benchmark results expose a profound gap between explicit constraints and implicit generalization. While PG-CD and PD-CD successfully project the trajectory to satisfy the explicit circular obstacle constraint $\Pi_{\mathcal{F}}$, this rigidity comes at a cost. Explicitly constrained architectures often incur higher average wall violation counts than the unconstrained baseline (e.g., PG-CD Stage 1 at 428.5 vs. UCD Stage 1 at 423.8; PD-CD Stage 2 at 541.5 vs. UCD Stage 2 at 502.5). By aggressively projecting waypoints away from the analytical obstacle, they force the robot closer to the implicit maze walls that they were never explicitly trained to avoid.

For deployment in continuous control, this indicates that explicit score-space projection architectures cannot be safely deployed if all relevant geometric boundaries are not included in the analytical projection function.

6.2 Relationship to the Constrained Diffuser Literature

The present findings constitute a critical *execution-level empirical validation* of the theoretical claims advanced in the Constrained Diffuser framework [14]. Whereas prior evaluations measured constraint satisfaction in waypoint space (offline metrics) against explicitly defined analytical boundaries, the present work measures it in action-space execution (online metrics) against the full geometry of the environment.

The empirical reality demonstrates that while explicit constraints strictly enforce analytical requirements, they fail to generalize to the implicitly learned background geometry. Deploying such planners safely requires either comprehensively defining all environmental constraints analytically—such as through Control Barrier Functions [3]—or fundamentally improving the diffusion prior's sensitivity to implicitly learned topological boundaries. This analysis heavily relied on the foundational Constrained Diffuser repository, while integrating and adapting execution-level paradigms from SafeDiffuser [13] to rectify unstable tracking dynamics.

6.3 Limitations and Open Problems

- **Horizon sensitivity and DP tolerance.** The Douglas–Peucker shortener is a post-hoc heuristic requiring manual tuning of ϵ_{DP}. A principled variable-horizon diffusion model would address FM-3 at the generative level, removing the need for post-processing altogether.

- **Two-dimensional setting.** All experiments are conducted in $\mathbb{R}^2$. The lookahead geometry (Section 4.1) and EMA analysis (Section 4.2) extend naturally to higher dimensions, but the effectiveness of the DP shortener on higher-dimensional trajectory manifolds remains untested.

- **IDM distributional shift.** The IDM f_ϕ is trained on unconstrained offline trajectories. Constrained diffusion may generate waypoints in regions of state space underrepresented in the training data, introducing action-prediction errors not captured by the present evaluation.

- **Curvature threshold.** The adaptive lookahead threshold κ_{thresh} is set heuristically. A learning-based or analytically derived curvature estimator would likely generalise better across different maze geometries.

- **Single-episode planning.** The diffusion model replans once per episode. Real-time replanning, e.g., at intervals of H steps, would permit recovery from mid-episode tracking failures, but at the cost of additional computational burden.

Chapter 7

Future Work

FW1. Variable-Horizon Diffusion Planners. A diffusion model conditioned on an estimated geodesic path length $L_{\text{phys}}(s_0, s_g)$ would permit the temporal horizon T to be adapted to the physical distance at inference, eliminating FM-3 at the generative level and removing the dependency on the Douglas–Peucker post-processor.

FW2. Model Predictive Extension of Pure-Pursuit. A receding-horizon MPC layer that re-solves for optimal lookahead scheduling at each step, using predicted future curvature, would replace the heuristic κ_{thresh} threshold with an optimally tuned adaptive gain.

FW3. Constraint-Aware IDM Training. The IDM training objective could be augmented with a constraint-violation penalty, $\mathcal{L}_{\text{IDM}} = \mathcal{L}_{\text{MSE}} + \mu \cdot c(s_{t+1})$, where $c(\cdot)$ penalises proximity to obstacle boundaries, introducing a second safety layer downstream of the planner.

FW4. Topological Homotopy Monitor. A real-time monitor computing the winding number of the generated path around each maze obstacle at the conclusion of denoising would permit detection of a homotopy-class flip relative to the shortest-path homotopy class, triggering a targeted re-denoising pass with increased Lagrangian penalty on the errant multiplier.

FW5. Sim-to-Real Transfer.

Deployment of the stabilised pipeline on a physical differential-drive robot in an indoor maze would allow validation of all three failure modes under sensor noise, wheel slip, and actuator delay.

Chapter 8

Conclusion

This report has set out a structured account of how three execution-level Cyber-Physical tracking failures were diagnosed and corrected in a Constrained Diffuser robotic navigation pipeline deployed on the `maze2d-large-v1` benchmark.

The central finding is that deployment failures in fixed-horizon diffusion planners are predominantly *execution-layer* phenomena rather than model-quality failures. Three independent failure modes were identified, each at a distinct level of abstraction:

- **FM-1** (geometric): pursuit-vector collapse and corner-cutting resulting from an improperly tuned lookahead, resolved by deriving the feasibility interval analytically and adopting the geometric-mean-informed value $d_{\text{la}} = 0.15$.
- **FM-2** (control-theoretic): limit-cycle oscillation resulting from EMA phase lag inside the feedback loop, resolved by removing the filter from the control path and adopting a curvature-aware adaptive lookahead schedule, reducing execution steps by 15.5% (490 $\rightarrow$ 414).
- **FM-3** (generative-model): distance burning and homotopy collapse resulting from the fixed-horizon temporal prior, resolved by post-hoc Douglas–Peucker path shortening applied upstream of the tracking controller.

Under the fully stabilised pipeline, the empirical benchmark yielded an unexpected insight into the limits of explicitly constrained architectures. While PG-CD and PD-CD successfully enforced the strict mathematical boundaries they were explicitly conditioned upon, this architectural rigidity compromised their generalized safety. When deployed in complex environments with implicitly learned boundaries (the maze walls), the aggressive explicit projections forced the controllers closer to the background geometry, incurring higher boundary violations in many configurations (e.g., PG-CD Stage 1 at 428.5 vs. UCD Stage 1 at 423.8; PD-CD Stage 2 at 541.5 vs. UCD Stage 2 at 502.5) than the unconstrained UCD baseline. Explicit constraint satisfaction, paradoxically, degraded overall safety.

None of the three fixes developed in this report—adaptive Pure-Pursuit scheduling,

the EMA-bypass protocol, and Tier-1 Douglas–Peucker shortening—depends on the particular diffusion architecture being used, and all three should apply directly to other fixed-horizon planners deployed in continuous Cyber-Physical control settings. It is hoped that this work contributes, in some small part, to closing the gap between planning and execution in generative robotic systems.

Bibliography

[1] Joshua Achiam, David Held, Aviv Tamar, and Pieter Abbeel. Constrained policy optimization. In *International Conference on Machine Learning (ICML)*, 2017.

[2] Anurag Ajay, Yilun Du, Abhi Gupta, Joshua B. Tenenbaum, Tommi Jaakkola, and Pulkit Agrawal. Is conditional generative modeling all you need for decision-making? In *International Conference on Learning Representations (ICLR)*, 2023.

[3] Aaron D. Ames, Samuel Coogan, Magnus Egerstedt, Gennaro Notomista, Koushil Sreenath, and Paulo Tabuada. Control barrier functions: Theory and applications. In *2019 18th European Control Conference (ECC)*, pages 3420–3431, 2019. doi: 10.23919/ECC.2019.8796030.

[4] R. Craig Coulter. Implementation of the pure pursuit path tracking algorithm. Technical report, Carnegie Mellon University, Robotics Institute, 1992.

[5] David H Douglas and Thomas K Peucker. Algorithms for the reduction of the number of points required to represent a digitized line or its caricature. *Cartographica: The International Journal for Geographic Information and Geovisualization*, 10(2):112–122, 1973.

[6] Justin Fu, Aviral Kumar, Ofir Nachum, George Tucker, and Sergey Levine. D4rl: Datasets for deep data-driven reinforcement learning. *arXiv preprint arXiv:2004.07219*, 2020.

[7] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[8] Michael Janner, Yilun Du, Joshua B. Tenenbaum, and Sergey Levine. Planning with diffusion for flexible behavior synthesis. In *International Conference on Machine Learning (ICML)*, 2022.

[9] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.

[10] Jean-Claude Latombe. *Robot Motion Planning*. Kluwer Academic Publishers, 1991.

[11] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. *International Conference on Learning Representations (ICLR)*, 2019.

[12] Diganta Misra. Mish: A self regularized non-monotonic neural activation function. *arXiv preprint arXiv:1908.08681*, 2019.

[13] Wei Xiao, Tsun-Hsuan Wang, Chuang Gan, and Daniela Rus. Safediffuser: Safe planning with diffusion probabilistic models. *arXiv preprint arXiv:2306.00148*, 2023. https://arxiv.org/pdf/2306.00148, https://github.com/Weixy21/SafeDiffuser.

[14] Jichen Zhang, Liqun Zhao, Antonis Papachristodoulou, and Jack Umenberger. Constrained diffusers for safe planning and control. In *Advances in Neural Information Processing Systems*, volume 38, pages 34965–34998, 2026. https://arxiv.org/pdf/2506.12544, https://github.com/z7076/Constrained_Diffuser.

Appendix

Appendix A

Lookahead Geometry: Formal Derivation

Closed-Loop Transfer Function of Pure-Pursuit

The point-mass robot is modelled as an integrator plant:

$$P(z) = \frac{1}{z-1} \quad \text{(unit-delay integrator in discrete time).} \tag{A.1}$$

The Pure-Pursuit controller computes a proportional position command with effective gain $K = 2/d_{\text{la}}$, obtained by linearising Equation (2.4) for small α:

$$K_{\text{pp}}(z) = \frac{2}{d_{\text{la}}}. \tag{A.2}$$

The open-loop transfer function is

$$G_{\text{ol}}(z) = K_{\text{pp}} \cdot P(z) = \frac{2/d_{\text{la}}}{z-1}. \tag{A.3}$$

The closed-loop poles are located at

$$z = 1 - \frac{2}{d_{\text{la}}}. \tag{A.4}$$

Stability requires $|z| < 1$, i.e., $-1 < 1 - 2/d_{\text{la}} < 1$, giving $d_{\text{la}} > 1$ under the idealised integrator model. In the physical Pure-Pursuit setting, where waypoint spacing Δw is finite, the analogous condition is $d_{\text{la}} > \epsilon_{\text{track}}$, recovering the lower bound of Equation (4.1).

Effect of EMA on Phase Margin

Introducing the EMA (Equation (4.5)) into the forward path replaces K_{pp} with $K_{\text{pp}} \cdot H(z)$:

$$G'_{\text{ol}}(z) = \frac{2\alpha}{d_{\text{la}}} \cdot \frac{z}{(z-1)(z-(1-\alpha))}. \tag{A.5}$$

This introduces an additional pole at $z = 1 - \alpha < 1$, which (i) increases the gain crossover frequency, and (ii) reduces the phase at crossover by $|\phi(\omega^*)|$ (Equation (4.6)). For $\alpha = 0.9$, a typical smoothing coefficient, the resulting phase margin reduction at $\omega^* = \pi/4$ is approximately $-18°$, consistent with the underdamped response observed in Figure 4.2.

Appendix B

Douglas–Peucker Algorithm (Full Pseudocode)

Algorithm 2 Douglas–Peucker Path Simplification

Input: Point sequence $P = [p_0, p_1, \ldots, p_n]$, tolerance $\epsilon_{\mathrm{DP}} > 0$

Output: Simplified sequence $P' \subseteq P$

function DouglasPeucker(P, ϵ_{DP})
  $d_{\max} \leftarrow 0, \quad k \leftarrow 0$
  for $i \leftarrow 1$ **to** $n-1$ **do**
    $d \leftarrow \mathrm{PerpDist}(p_i, p_0, p_n)$
    if $d > d_{\max}$ **then**
      $k \leftarrow i; \quad d_{\max} \leftarrow d$
    end if
  end for
  if $d_{\max} > \epsilon_{\mathrm{DP}}$ **then**
    $L \leftarrow \mathrm{DouglasPeucker}(P[0:k], \epsilon_{\mathrm{DP}})$
    $R \leftarrow \mathrm{DouglasPeucker}(P[k:n], \epsilon_{\mathrm{DP}})$
    return $L[:-1] \cup R$
  else
    return $[p_0, \ p_n]$
  end if
end function

function PerpDist(p, p_0, p_n)
  $\mathbf{u} \leftarrow p_n - p_0; \quad \hat{\mathbf{u}} \leftarrow \mathbf{u} / \|\mathbf{u}\|$
  return $\left\| (p - p_0) - \left[(p - p_0) \cdot \hat{\mathbf{u}} \right] \hat{\mathbf{u}} \right\|$
end function

Appendix C

Hyperparameter and Configuration Reference

Table C.1: Complete hyperparameter configuration for all experiments.

Parameter	Description	Value
T	Diffusion horizon	384
$d_{\text{la}}^{\text{base}}$	Base lookahead distance	0.15
$d_{\text{la}}^{\text{straight}}$	Lookahead on straight segments	0.30
$d_{\text{la}}^{\text{curve}}$	Lookahead on curved segments	0.15
κ_{thresh}	Curvature threshold (cosine)	0.95
r_{obs}	Obstacle clearance radius	0.50
δ_{goal}	Goal tolerance	0.50
ϵ_{track}	Measured max tracking error	0.10
ϵ_{DP}	Douglas–Peucker tolerance	0.25
η (PD)	Dual multiplier step size	2.5×10^{-2}
α (EMA, disabled)	EMA coefficient	N/A
IDM hidden size	MLP hidden layer width	512
IDM training	Max epochs / batch size	10 / 2048